```
#logistic regression
#classification for data types
```

```
import seaborn as sns
df=sns.load_dataset("titanic")
df
```

| | survived | pclass | sex | age | sibsp | parch | fare | embarked | class | who | adult_male | deck | embark_town | alive | alo |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 3 | male | 22.0 | 1 | 0 | 7.2500 | S | Third | man | True | NaN | Southampton | no | Fa |
| **1** | 1 | 1 | female | 38.0 | 1 | 0 | 71.2833 | C | First | woman | False | C | Cherbourg | yes | Fa |
| **2** | 1 | 3 | female | 26.0 | 0 | 0 | 7.9250 | S | Third | woman | False | NaN | Southampton | yes | T |
| **3** | 1 | 1 | female | 35.0 | 1 | 0 | 53.1000 | S | First | woman | False | C | Southampton | yes | Fa |
| **4** | 0 | 3 | male | 35.0 | 0 | 0 | 8.0500 | S | Third | man | True | NaN | Southampton | no | T |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | |
| **886** | 0 | 2 | male | 27.0 | 0 | 0 | 13.0000 | S | Second | man | True | NaN | Southampton | no | T |
| **887** | 1 | 1 | female | 19.0 | 0 | 0 | 30.0000 | S | First | woman | False | B | Southampton | yes | T |
| **888** | 0 | 3 | female | NaN | 1 | 2 | 23.4500 | S | Third | woman | False | NaN | Southampton | no | Fa |
| **889** | 1 | 1 | male | 26.0 | 0 | 0 | 30.0000 | C | First | man | True | C | Cherbourg | yes | T |
| **890** | 0 | 3 | male | 32.0 | 0 | 0 | 7.7500 | Q | Third | man | True | NaN | Queenstown | no | T |

891 rows × 15 columns

Next steps: Generate code with df New interactive sheet

```
df['age']=df['age'].fillna(df['age'].mean())
df
```

| | survived | pclass | sex | age | sibsp | parch | fare | embarked | class | who | adult_male | deck | embark_town | alive |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 3 | male | 22.000000 | 1 | 0 | 7.2500 | S | Third | man | True | NaN | Southampton | no |
| **1** | 1 | 1 | female | 38.000000 | 1 | 0 | 71.2833 | C | First | woman | False | C | Cherbourg | yes |
| **2** | 1 | 3 | female | 26.000000 | 0 | 0 | 7.9250 | S | Third | woman | False | NaN | Southampton | yes |
| **3** | 1 | 1 | female | 35.000000 | 1 | 0 | 53.1000 | S | First | woman | False | C | Southampton | yes |
| **4** | 0 | 3 | male | 35.000000 | 0 | 0 | 8.0500 | S | Third | man | True | NaN | Southampton | no |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| **886** | 0 | 2 | male | 27.000000 | 0 | 0 | 13.0000 | S | Second | man | True | NaN | Southampton | no |
| **887** | 1 | 1 | female | 19.000000 | 0 | 0 | 30.0000 | S | First | woman | False | B | Southampton | yes |
| **888** | 0 | 3 | female | 29.699118 | 1 | 2 | 23.4500 | S | Third | woman | False | NaN | Southampton | no |
| **889** | 1 | 1 | male | 26.000000 | 0 | 0 | 30.0000 | C | First | man | True | C | Cherbourg | yes |
| **890** | 0 | 3 | male | 32.000000 | 0 | 0 | 7.7500 | Q | Third | man | True | NaN | Queenstown | no |

891 rows × 15 columns

Next steps: Generate code with df New interactive sheet

```
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
a=["sex","who","class","embarked","adult_male","deck","embark_town","alive","alone"]
for i in a:
  df[i]=le.fit_transform(df[i])
df
```

| | survived | pclass | sex | age | sibsp | parch | fare | embarked | class | who | adult_male | deck | embark_town | alive | alon |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 3 | 1 | 22.000000 | 1 | 0 | 7.2500 | 2 | 2 | 1 | 1 | 7 | 2 | 0 | |
| **1** | 1 | 1 | 0 | 38.000000 | 1 | 0 | 71.2833 | 0 | 0 | 2 | 0 | 2 | 0 | 1 | |
| **2** | 1 | 3 | 0 | 26.000000 | 0 | 0 | 7.9250 | 2 | 2 | 2 | 0 | 7 | 2 | 1 | |
| **3** | 1 | 1 | 0 | 35.000000 | 1 | 0 | 53.1000 | 2 | 0 | 2 | 0 | 2 | 2 | 1 | |
| **4** | 0 | 3 | 1 | 35.000000 | 0 | 0 | 8.0500 | 2 | 2 | 1 | 1 | 7 | 2 | 0 | |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | . |
| **886** | 0 | 2 | 1 | 27.000000 | 0 | 0 | 13.0000 | 2 | 1 | 1 | 1 | 7 | 2 | 0 | |
| **887** | 1 | 1 | 0 | 19.000000 | 0 | 0 | 30.0000 | 2 | 0 | 2 | 0 | 1 | 2 | 1 | |
| **888** | 0 | 3 | 0 | 29.699118 | 1 | 2 | 23.4500 | 2 | 2 | 2 | 0 | 7 | 2 | 0 | |
| **889** | 1 | 1 | 1 | 26.000000 | 0 | 0 | 30.0000 | 0 | 0 | 1 | 1 | 2 | 0 | 1 | |
| **890** | 0 | 3 | 1 | 32.000000 | 0 | 0 | 7.7500 | 1 | 2 | 1 | 1 | 7 | 1 | 0 | |

891 rows × 15 columns

Next steps: Generate code with df New interactive sheet

```
x=df.drop("survived",axis=1)
y=df["survived"]
x
```

| | pclass | sex | age | sibsp | parch | fare | embarked | class | who | adult_male | deck | embark_town | alive | alone |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 3 | 1 | 22.000000 | 1 | 0 | 7.2500 | 2 | 2 | 1 | 1 | 7 | 2 | 0 | 0 |
| **1** | 1 | 0 | 38.000000 | 1 | 0 | 71.2833 | 0 | 0 | 2 | 0 | 2 | 0 | 1 | 0 |
| **2** | 3 | 0 | 26.000000 | 0 | 0 | 7.9250 | 2 | 2 | 2 | 0 | 7 | 2 | 1 | 1 |
| **3** | 1 | 0 | 35.000000 | 1 | 0 | 53.1000 | 2 | 0 | 2 | 0 | 2 | 2 | 1 | 0 |
| **4** | 3 | 1 | 35.000000 | 0 | 0 | 8.0500 | 2 | 2 | 1 | 1 | 7 | 2 | 0 | 1 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| **886** | 2 | 1 | 27.000000 | 0 | 0 | 13.0000 | 2 | 1 | 1 | 1 | 7 | 2 | 0 | 1 |
| **887** | 1 | 0 | 19.000000 | 0 | 0 | 30.0000 | 2 | 0 | 2 | 0 | 1 | 2 | 1 | 1 |
| **888** | 3 | 0 | 29.699118 | 1 | 2 | 23.4500 | 2 | 2 | 2 | 0 | 7 | 2 | 0 | 0 |
| **889** | 1 | 1 | 26.000000 | 0 | 0 | 30.0000 | 0 | 0 | 1 | 1 | 2 | 0 | 1 | 1 |
| **890** | 3 | 1 | 32.000000 | 0 | 0 | 7.7500 | 1 | 2 | 1 | 1 | 7 | 1 | 0 | 1 |

891 rows × 14 columns

Next steps: Generate code with x New interactive sheet

```
x.drop(["alive","alone","embark_town","deck","class","who","adult_male"],axis=1,inplace = True)
x.isnull().sum()
x.dropna()
y.dropna()
```

| | survived |
|---|---|
| **0** | 0 |
| **1** | 1 |
| **2** | 1 |
| **3** | 1 |
| **4** | 0 |
| **...** | ... |
| **886** | 0 |
| **887** | 1 |
| **888** | 0 |
| **889** | 1 |
| **890** | 0 |

891 rows × 1 columns

**dtype:** int64

```
from sklearn.model_selection import train_test_split

x_train,x_test,y_train,y_test=train_test_split(x,y ,test_size=0.2,random_state=42)
```

```
from sklearn.linear_model import LogisticRegression
model=LogisticRegression()
model.fit(x_train,y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

```
▾ LogisticRegression
LogisticRegression()
```

```
y_pred = model.predict(x_test)

y_prob = model.predict_proba(x_test)
```

```
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.8100558659217877
```

```
from sklearn.metrics import confusion_matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print(conf_matrix)
    #[[true positive    false positive ]
   #  [false negative   true negative]]
```

```
[[90 15]
 [19 55]]
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.83      0.86      0.84       105
           1       0.79      0.74      0.76        74

    accuracy                           0.81       179
   macro avg       0.81      0.80      0.80       179
```

```
weighted avg       0.81      0.81      0.81       179
```

```
from matplotlib import pyplot as plt
import seaborn as sns
sns.scatterplot(y,color ="red")
sns.lineplot(y_pred,color ="blue")
plt.show()
```